

Big Data Programming

Function

Chapter Overview

이번 시간의 학습 흐름

함수를 정의하고 호출한다

내장 함수를 사용하는 방법에서 출발하여 직접 함수를 만든다. def를 실행하는 시점과 함수 본문을 실행하는 시점을 구분한다.

입력과 결과가 이동하는 과정을 읽는다

인수와 매개변수, print()와 return의 역할을 구분한다. 주사위와 문자열 생성 예제를 통해 같은 함수를 여러 번 사용하는 이유를 확인한다.

함수 안팎의 이름과 출력 형식을 다룬다

문자열과 리스트를 전달했을 때의 차이, 지역·전역 변수, 기본값과 키워드 인수를 살펴본다. 마지막에는 반환값을 원하는 문자열 형식으로 표현한다.

Python Functions

함수는 반복되는 작업에 이름을 붙인다

비슷한 코드를 여러 번 복사하지 않는다

1부터 10까지의 합과 1부터 20까지의 합은 끝값만 다르다. 공통 계산을 함수로 묶으면 바뀌는 값만 전달하여 같은 작업을 다시 사용할 수 있다.

입력과 처리 규칙을 분리한다

start와 end는 호출할 때 정하고, 누적하는 규칙은 함수 안에 둔다. 계산 규칙을 고칠 때에도 같은 코드를 여러 곳에서 수정할 필요가 줄어든다.

함수 이름은 수행할 일을 드러낸다

get_sum(1, 10)은 합계를 구하는 호출임을 보여 준다. 함수는 단순히 줄 수를 줄이는 장치가 아니라 프로그램을 의미 있는 작업 단위로 나누는 방법이다.

같은 함수를 다른 입력으로 사용

```
def get_sum(start, end):
    total = 0
    for i in range(start, end + 1):
        total += i
    return total


print(get_sum(1, 10))
print(get_sum(1, 20))
```

실행 결과

```
55
210
```

Python Functions

내장 함수와 직접 정의한 함수

내장 함수는 Python이 기본 제공한다

print(), max(), min(), len()처럼 바로 사용하는 내장 함수가 있다. int와 float는 자료형을 나타내는 내장 이름이며, 호출하면 변환 등에 사용할 수 있다.

사용자 정의 함수는 def로 만든다

필요한 계산이나 작업을 직접 작성하고 이름을 붙인다. 어떤 값을 받아 무엇을 돌려줄지 정하면 다른 코드에서 그 세부 과정을 반복해 작성하지 않아도 된다.

내장 이름을 변수로 가리지 않는다

max는 예약어는 아니지만 max = 10으로 대입하면 해당 이름으로 내장 함수를 호출하기 어렵다. 합계에는 total처럼 역할이 드러나는 다른 이름을 사용한다.

지금까지 사용한 내장 기능

```
print(len("Python"))
print(max(3, 5, 2))
print(int("20"))
```

실행 결과

```
6
5
20
```

모듈에서 가져오는 함수

random.randrange()는 random 모듈의 함수이다. 먼저 import random으로 모듈을 불러와 사용한다.

Built-in Functions

max()와 min()은 무엇을 비교할까?

문자열을 전달하면 문자들을 비교한다

max("Hello world")는 문자열 전체를 큰 값 하나로 바꾸는 것이 아니라, 그 안에서 가장 큰 문자를 찾는다. 이 문자열의 결과는 소문자 w이다.

공백도 문자열을 구성하는 문자이다

min("Hello world")의 결과는 공백 한 칸이다. print(tiny)만 사용하면 빈 출력처럼 보이므로 repr(tiny)로 따옴표 안의 공백을 확인한다.

비교 기준과 출력 모양을 구분한다

문자열의 순서는 문자의 유니코드 코드 포인트를 기준으로 한다. 알파벳만 있다는 가정이나 대소문자를 무시하는 사전 순서를 자동 적용하지 않는다.

문자열에서 가장 큰 문자와 작은 문자

```
big = max("Hello world")
tiny = min("Hello world")
print(big)
print(repr(tiny))
```

실행 결과

```
w
' '
```

숫자의 비교와 구분

max(3, 5, 2)는 5이고 min(3, 5, 2)는 2이다. 무엇을 인수로 전달했는지 먼저 확인한다.

Built-in Functions

DEC	HEX	OCT	Char	DEC	HEX	OCT	Char	DEC	HEX	OCT	Char
0	00	000	Ctrl-@ NUL	43	2B	053	+	86	56	126	V
1	01	001	Ctrl-A SOH	44	2C	054	,	87	57	127	W
2	02	002	Ctrl-B STX	45	2D	055	-	88	58	130	X
3	03	003	Ctrl-C ETX	46	2E	056	.	89	59	131	Y
4	04	004	Ctrl-D EOT	47	2F	057	/	90	5A	132	Z
5	05	005	Ctrl-E ENQ	48	30	060	0	91	5B	133	[
6	06	006	Ctrl-F ACK	49	31	061	1	92	5C	134	₩
7	07	007	Ctrl-G BEL	50	32	062	2	93	5D	135	]
8	08	010	Ctrl-H BS	51	33	063	3	94	5E	136	^
9	09	011	Ctrl-I HT	52	34	064	4	95	5F	137	_
10	0A	012	Ctrl-J LF	53	35	065	5	96	60	140	`
11	0B	013	Ctrl-K VT	54	36	066	6	97	61	141	a
12	0C	014	Ctrl-L FF	55	37	067	7	98	62	142	b
13	0D	015	Ctrl-M CR	56	38	070	8	99	63	143	c
14	0E	016	Ctrl-N SO	57	39	071	9	100	64	144	d
15	0F	017	Ctrl-O SI	58	3A	072	:	101	65	145	e
16	10	020	Ctrl-P DLE	59	3B	073	;	102	66	146	f
17	11	021	Ctrl-Q DCI	60	3C	074	<	103	67	147	g
18	12	022	Ctrl-R DC2	61	3D	075	=	104	68	150	h
19	13	023	Ctrl-S DC3	62	3E	076	>	105	69	151	i
20	14	024	Ctrl-T DC4	63	3F	077	?	106	6A	152	j
21	15	025	Ctrl-U NAK	64	40	100	@	107	6B	153	k
22	16	026	Ctrl-V SYN	65	41	101	A	108	6C	154	l
23	17	027	Ctrl-W ETB	66	42	102	B	109	6D	155	m
24	18	030	Ctrl-X CAN	67	43	103	C	110	6E	156	n
25	19	031	Ctrl-Y EM	68	44	104	D	111	6F	157	o
26	1A	032	Ctrl-Z SUB	69	45	105	E	112	70	160	p
27	1B	033	Ctrl-[ESC	70	46	106	F	113	71	161	q
28	1C	034	Ctrl-₩ FS	71	47	107	G	114	72	162	r
29	1D	035	Ctrl-] GS	72	48	110	H	115	73	163	s
30	1E	036	Ctrl-^ RS	73	49	111	I	116	74	164	t
31	1F	037	Ctrl_ US	74	4A	112	J	117	75	165	u
32	20	040	Space	75	4B	113	K	118	76	166	v
33	21	041	!	76	4C	114	L	119	77	167	w
34	22	042	"	77	4D	115	M	120	78	170	x
35	23	043	#	78	4E	116	N	121	79	171	y
36	24	044	$	79	4F	117	O	122	7A	172	z
37	25	045	%	80	50	120	P	123	7B	173	{
38	26	046	&	81	51	121	Q	124	7C	174	|
39	27	047	'	82	52	122	R	125	7D	175	}
40	28	050	(	83	53	123	S	126	7E	176	

교할까?

한다

를 큰 값 하나로
장 큰 문자를
w이다.

다

한 칸이다.
보이므로
인한다.

다

코드 포인트를
정이나
자동 적용하지

문자열에서 가장 큰 문자와 작은 문자

```
big = max("Hello world")
tiny = min("Hello world")
print(big)
print(repr(tiny))
```

실행 결과

```
w
' '
```

숫자의 비교와 구분

max(3, 5, 2)는 5이고 min(3, 5, 2)는 2이다. 무엇을 인수로 전달했는지 먼저 확인한다.

Random-Number Generation

주사위는 1 이상 7 미만의 정수로 표현한다

randrange()는 범위에서 정수를 선택한다

random.randrange(1, 7)은 1부터 6까지 중 하나를 돌려준다. 시작값은 포함하고 끝값 7은 포함하지 않으므로 여섯 면 주사위의 눈을 표현할 수 있다.

함수 호출과 반복의 역할이 다르다

randrange() 한 번은 값 하나를 구한다. for문이 같은 호출을 열 번 실행하고 print()의 end=" "가 값을 한 줄에 표시하게 한다.

가능한 범위와 실제 출력은 구분한다

실행할 때마다 출력 순서가 달라질 수 있으며 같은 눈이 연속으로 나올 수도 있다. 열 번 실행한다고 1부터 6까지가 모두 한 번씩 나타나는 것은 아니다.

여섯 면 주사위를 열 번 던지기

```
import random

for roll in range(10):
    print(random.randrange(1, 7),
          end=" ")
```

결과를 읽을 때 확인할 내용

출력된 값은 열 개여야 한다. 각 값이 1 이상 6 이하인지 확인하며, 다음 실행에서 같은 순서가 재현될 것으로 가정하지 않는다.

사용 범위

random은 수업용 모의실험에 사용한다. 보안 비밀번호나 인증 토큰 생성용 난수와는 구분한다.

Python Functions

Notebook에서 반환값과 난수 출력 확인하기

In [1]:
```
big = max("Hello world")
tiny = min("Hello world")
print(big)
print(repr(tiny))
```

```
w
' '
```

In [2]:
```
import random
for roll in range(10):
    print(random.randrange(1, 7), end=" ")
```

```
4 2 1 4 4 5 2 4 4 6
```

실행한 Notebook의 코드와 출력이다. max()는 소문자 w, min()은 공백을 반환한다. 아래쪽 난수 출력은 가능한 한 실행 예이며 다음 실행에서는 순서가 달라질 수 있다.

Defining Functions

def, 매개변수, 들여쓰기, return 읽기

첫 줄은 함수의 이름과 입력을 정한다

def addtwo(a, b):에서 def는 정의를 시작하는 키워드이고 a와 b는 매개변수이다. 콜론 다음의 들여쓴 문장들이 함수 본문을 이룬다.

들여쓰기는 함수의 경계를 나타낸다

본문 안의 문장은 같은 단계로 들여쓴다. 들여쓰기가 끝난 print(addtwo(3, 5))는 함수 밖에 있으며, 함수를 실제로 호출하는 문장이다.

설명 문자열과 반환문을 구분한다

본문 첫 문장의 문자열은 함수 설명인 docstring으로 사용할 수 있다. return은 값을 호출한 곳에 돌려주며, 설명을 작성하는 기능과 다르다.

정의의 각 부분을 확인하는 예

```
def addtwo(a, b):
    """두 값을 더하여 반환한다."""
    added = a + b
    return added

print(addtwo(3, 5))
```

실행 결과

```
8
```

함수 밖의 코드

마지막 print()는 함수 본문이 아니다. 이 호출이 있어야 addtwo의 계산이 수행되고 결과를 확인할 수 있다.

Defining Functions

정의만 했을 때와 호출했을 때

정의는 재사용할 함수를 준비한다

def 문을 실행하면 이름 print_address에 함수가 연결된다. 이때 본문에 있는 인사말 출력은 아직 실행되지 않는다.

호출은 본문의 작업을 실행한다

print_address()를 만나면 함수 안의 print()가 실행된다. 같은 호출을 두 번 적으면 본문도 두 번 실행되며, 함수 정의를 다시 복사할 필요가 없다.

함수 정의와 두 번의 호출

```
def print_address():
    print("I sleep all night and I work all day")

print_address()
print_address()
```

실행 결과

```
I sleep all night and I work all day
I sleep all night and I work all day
```

Defining Functions

정의 셀에는 출력이 없고 호출 셀에는 출력이 있다

In [1]:
```
def print_address():
    print("I sleep all night and I work all day")
```

In [2]:
```
print_address()
print_address()
```

```
I sleep all night and I work all day
I sleep all night and I work all day
```

정의 셀을 실행해도 인사말은 아직 출력되지 않는다. 다음 셀에서 같은 함수를 두 번 호출하면 본문이 두 번 실행되어 인사말 두 줄이 출력된다.

Arguments and Parameters

인수와 매개변수는 위치와 역할이 다르다

매개변수는 정의 쪽의 이름이다

def addtwo(a, b):에서 a와 b는 함수가 입력을 사용할 때 참조할 이름이다. 함수 안에서는 이 이름들을 이용하여 계산을 작성한다.

인수는 호출 쪽에서 전달하는 값이다

addtwo(3, 5)의 3과 5는 인수이다. 호출이 시작되면 이번 호출의 a는 3을, b는 5를 가리키게 된다. 다음 호출에는 다른 값을 전달할 수 있다.

이름 자체가 같아야 하는 것은 아니다

호출하는 코드의 변수명이 x와 y여도 함수 안에서는 a와 b를 사용한다. 기본적인 위치 인수는 전달한 순서에 따라 매개변수에 대응한다.

정의하는 쪽

```
def addtwo(a, b):
    return a + b
```

호출하는 쪽

```
x, y = 3, 5
answer = addtwo(x, y)
print(answer)
```

이번 호출의 대응

첫 번째 인수 x의 값 3 → a
두 번째 인수 y의 값 5 → b
반환값 8 → answer

Arguments and Parameters

greet(lang): 입력에 따라 다른 경로 실행하기

하나의 함수가 여러 경우를 처리한다

lang은 이번 호출에서 받은 언어 코드를 뜻한다. es이면 Hola, fr이면 Bonjour, 그 밖의 값이면 Hello를 출력한다.

조건문은 호출할 때마다 새로 판단한다

greet("en")과 greet("es")는 같은 함수 본문을 사용하지만 lang의 값이 다르다. 하나의 if/elif/else에서 선택된 경로 하나를 실행한다.

이 예제는 인사말을 화면에 출력한다

현재 greet는 인사말 문자열을 return하지 않는다. 값을 계산에 이어서 사용하려는 경우와 화면에 보여주려는 경우를 다음 예제에서 비교한다.

언어 코드에 따른 인사

```
def greet(lang):
    if lang == "es":
        print("Hola")
    elif lang == "fr":
        print("Bonjour")
    else:
        print("Hello")


greet("en")
greet("es")
greet("fr")
```

실행 결과

순서대로 Hello, Hola, Bonjour를 출력한다.

Return Values

return은 호출한 곳에서 사용할 값을 돌려준다

반환값은 함수 호출 표현식의 결과이다

greet()가 "Hello"를 반환하면 호출한 곳은 그 값을 다른 문자열과 함께 출력할 수 있다. 함수가 인사말을 직접 출력하는 구조와 구분한다.

돌려받은 값은 저장하거나 다시 사용한다

같은 반환값을 Glenn과 Sally라는 이름에 각각 연결한다. 호출 결과를 변수에 대입하면 이후 계산이나 다른 함수의 입력에도 사용할 수 있다.

같은 이름을 다시 정의하면 이전 정의가 바뀐다

앞 예제는 greet(lang)이지만 여기서는 인수가 없는 greet()이다. Notebook에서 이 정의를 실행하면 이름 greet는 새 함수를 가리키므로 두 예제를 구분한다.

인사말을 반환하는 별도의 예제

```
def greet():
    return "Hello"


print(greet(), "Glenn")
print(greet(), "Sally")
```

실행 결과

```
Hello Glenn
Hello Sally
```

반환값을 변수로 받기

```
message = greet()
print(message)
```

Return Values

화면에 출력된 값과 반환된 값은 같지 않다

print()는 화면에 보이게 한다

show_sum()은 8을 출력하지만 계산 결과를 return하지 않는다. 호출 결과를 x에 받으면 x의 값은 8이 아니라 None이다.

return은 호출한 코드에 값을 전달한다

get_sum()은 8을 돌려준다. 이 값을 y에 저장한 뒤 출력하거나 y * 2 같은 추가 계산에 사용할 수 있다.

return이 없거나 값 없이 return하면 None이다

화면에 8이 보였다는 사실만으로 함수를 이용한 계산이 완성된 것은 아니다. 반환값을 이용하려면 그 값이 실제로 전달되는지 확인한다.

출력하는 함수와 반환하는 함수

```
def show_sum(a, b):
    print(a + b)

def get_sum(a, b):
    return a + b

x = show_sum(3, 5)
y = get_sum(3, 5)
print(x, y)
```

실행 결과

```
8
None 8
```

Return Values

같은 8이라도 출력과 반환의 경로가 다르다

In [2]:
```
x = show_sum(3, 5)
y = get_sum(3, 5)
```

```
8
```

In [3]:
```
print(x, y)
print(y * 2)
```

```
None 8
16
```

첫 셀의 8은 show_sum() 내부의 출력이다. 반환값을 받은 x는 None이고, get_sum()의 반환값을 받은 y는 8이다. y를 이용한 다음 계산은 16을 만든다.

Multiple Parameters

addtwo(3, 5)의 실행 과정을 따라가기

호출 지점에서 함수 본문으로 이동한다

x = addtwo(3, 5)를 실행할 때 먼저 addtwo를 호출한다. 이번 호출에서 a와 b는 각각 3과 5를 가리킨다.

지역 이름으로 계산하고 결과를 반환한다

added = a + b가 8을 계산한다. return added는 added라는 이름을 보내는 것이 아니라, 그 이름이 가리키는 값 8을 돌려준다.

호출한 곳의 대입이 완성된다

함수 호출이 8로 평가되면 바깥쪽의 x = ...가 완료된다. print(x)는 함수가 반환한 결과를 보여주며, added를 직접 사용하는 문장이 아니다.

예제 코드

```
def addtwo(a, b):
    added = a + b
    return added

x = addtwo(3, 5)
print(x)
```

값이 이동하는 순서

위치	이름 또는 식	값
함수 입력	a, b	3, 5
함수 내부	added	8
호출 결과	addtwo(3, 5)	8
함수 밖	x	8

Function Calls

호출하기 전에 정의가 실행되어 있어야 한다

새 실행 환경에서는 위에서부터 읽는다

print(power(10, 2))를 def power보다 먼저 실행하면 power라는 이름을 아직 찾을 수 없어 NameError가 발생한다.

정의를 먼저 실행한 뒤 호출한다

아래처럼 power를 정의하고 마지막 줄에서 호출하면 곱셈을 두 번 수행하여 100을 반환한다. 여기서는 y가 0 이상의 정수라고 가정한다.

Notebook에서는 이전 정의가 문제를 가릴 수 있다

위쪽 셀을 실행하지 않아도 과거에 정의한 함수가 커널에 남아 있으면 호출이 될 수 있다. 재시작 후 위에서부터 실행되는지 확인한다.

정상적인 순서

```
def power(x, y):
    result = 1
    for i in range(y):
        result = result * x
    return result

print(power(10, 2))
```

실행 결과

100

y가 0이라면

range(0)의 반복은 실행되지 않는다. 초기값 result = 1이 그대로 반환된다.

Return Values

return의 위치가 함수의 종료 시점을 정한다

return을 실행하면 그 호출은 끝난다

반환문 다음의 문장들은 같은 호출에서 더 실행되지 않는다. 반복문 안에 return을 넣으면 반복을 한 번만 수행하고 함수 전체가 끝날 수 있다.

계산이 끝난 뒤 반환할 위치를 정한다

power에서 result를 반복해서 곱하려면 return result가 for문 바깥에 있어야 한다. 함수 본문 안과 반복문 본문 안을 들여쓰기로 구분한다.

들여쓰기를 바꾸어 결과를 예상한다

오른쪽 잘못된 예에서는 power(10, 2)가 첫 곱셈 뒤에 종료되어 10을 반환한다. 같은 문장을 사용해도 return의 위치가 달라지면 결과가 달라진다.

반복을 너무 일찍 끝내는 예

```
def power(x, y):
    result = 1
    for i in range(y):
        result = result * x
        return result


print(power(10, 2))
```

실행 결과

10

수정할 부분

return result의 들여쓰기를 for문과 같은 단계로 옮긴다. 반복을 모두 끝낸 뒤 100을 반환하게 된다.

Practice

실습: 여섯 글자 문자열을 세 번 생성하기

문자 후보와 길이를 먼저 정한다

영문 소문자와 숫자 중에서 문자를 선택한다. 함수 한 번은 길이가 6인 문자열 하나를 반환하고, 함수 밖에서는 세 번 호출하여 출력한다.

기존 반복문을 함수 안에 묶는다

빈 문자열에서 시작하여 문자를 하나씩 이어 붙인다. 후보 문자열의 길이로 난수 인덱스의 범위를 정하고, 해당 위치의 문자를 가져온다.

실제 인증용 비밀번호와 구분한다

이 예제는 난수와 함수 연습용이다. random으로 만든 여섯 글자 문자열을 실제 비밀번호나 일회용 인증번호로 사용하지 않는다.

완성할 함수의 핵심

```
def genPass():
    alphabet = ("abcdefghijklmnopqrstuvwxyz"
                + "0123456789")
    password = ""
    for i in range(6):
        index = _____
        password = _____
    return password
```

확인할 조건

반환값의 길이는 6이다. 각 문자가 alphabet에 들어 있다. 세 번 호출하더라도 반드시 서로 다른 문자열이 나오는 것은 아니다.

Practice

해설: 선택한 문자를 누적한 뒤 반환한다

인덱스는 0부터 후보 개수 직전까지이다

len(alphabet)은 36이다.
random.randrange(36)은 0부터 35까지 중 하나를 선택하므로 후보 문자열의 유효한 위치를 가리킨다.

누적 변수는 호출할 때마다 다시 시작한다

password = ""는 함수 본문에 있으므로 매 호출마다 빈 문자열로 시작한다. 여섯 번의 반복이 끝나면 만들어진 문자열 하나를 반환한다.

반복 횟수와 호출 횟수는 다르다

함수 안의 여섯 번 반복은 글자 수를 만든다. 함수 밖의 세 번 호출은 문자열 세 개를 만든다. 두 횟수의 역할을 구분한다.

완성 코드

```
import random

def genPass():
    alphabet = ("abcdefghijklmnopqrstuvwxyz"
                + "0123456789")
    password = ""
    for i in range(6):
        index = random.randrange(len(alphabet))
        password = password + alphabet[index]
    return password

print(genPass())
print(genPass())
print(genPass())
```

Practice

Notebook에서 세 번의 함수 호출 확인하기

In [2]:
```
print(genPass())
print(genPass())
print(genPass())
```

```
kwkfpp
uekn4l
tpejh0
```

앞서 정의한 genPass()를 세 번 호출한 실제 출력이다. 각 문자열의 길이와 후보 문자를 확인한다. 이 화면의 문자 순서는 한 실행 예이며 항상 같은 결과가 나오지는 않는다.

Practice

문자열의 길이는 반복하면서 어떻게 늘어날까?

한 번 반복할 때 한 글자가 늘어난다

시작할 때 password의 길이는 0이다. 선택한 글자를 하나씩 붙이므로 반복을 두 번 끝냈다면 길이는 2, 여섯 번 끝냈다면 길이는 6이다.

반복 후	추가한 문자	누적 문자열	길이
시작	없음	빈 문자열	0
1	q	q	1
2	5	q5	2
3	4	q54	3
6	v	q546zv	6

4번째와 5번째 반복에서 각각 6과 z를 추가한 것으로 가정한다.

return이 반복문 안에 있으면 무엇이 달라질까?

첫 글자를 추가한 직후 반환하면 길이가 1인 문자열만 얻는다. 함수가 마지막 결과를 돌려주어야 하는 시점과 반복의 경계를 함께 확인한다.

Passing Arguments

Python의 인수 전달은 객체와 이름으로 이해한다

호출하면 매개변수에 객체가 연결된다

함수의 매개변수는 전달받은 객체를 가리키는 지역 이름이다. 호출한 쪽의 이름과 함수 안의 이름이 잠시 같은 객체를 참조할 수 있다.

이름 재대입과 객체 변경은 다른 동작이다

함수 안에서 매개변수를 다른 객체에 연결해도 호출한 쪽의 이름이 자동으로 재대입되지는 않는다. 반면 공유한 변경 가능 객체를 수정하면 바깥에서도 그 변화를 볼 수 있다.

자료형마다 전달 방식이 바뀌는 것은 아니다

문자열은 값 전달이고 리스트는 참조 전달이라는 두 규칙으로 나누지 않는다. Python에서는 같은 이름 연결 원리와 객체의 변경 가능 여부로 예제를 설명한다.

호출 전

msg → 문자열 객체
numbers → 리스트 객체

호출 직후

s와 msg가 같은 문자열을 참조한다.
li와 numbers가 같은 리스트를 참조한다.

이후 확인할 질문

다른 객체에 이름을 다시 연결했는가?
아니면 공유한 객체의 내용을 바꾸었는가?

용어

공식 FAQ는 인수가 대입으로 전달된다고 설명한다. 객체 공유에 의한 호출로도 이해할 수 있다.

Passing Arguments

문자열: 지역 이름을 다시 연결한다

호출 직후에는 같은 문자열을 가리킨다

modify1(msg)를 호출하면 지역 이름 s는 msg가 가리키는 "Happy Birthday"를 참조한다. 원본 문자열이 함수에 들어가며 자동 복제된다고 볼 필요는 없다.

문자열의 +=는 새 문자열에 다시 연결한다

문자열은 그 자체의 내용을 바꾸는 방식으로 수정할 수 없다. s += "To You"는 이어 붙인 새 문자열을 구한 뒤 지역 이름 s에 연결한다.

바깥의 msg는 이전 문자열을 유지한다

msg에 새 값을 대입하지 않았으므로 함수 호출 전후의 출력은 같다. 바깥에서 새 문자열을 사용하려면 반환받은 값을 msg 등에 대입하도록 작성할 수 있다.

원래 문자열이 유지되는 예

```
def modify1(s):
    s += "To You"

msg = "Happy Birthday"
print("msg=", msg)
modify1(msg)
print("msg=", msg)
```

실행 결과

```
msg= Happy Birthday
msg= Happy Birthday
```

함수 안의 s

s는 Happy BirthdayTo You를 가리키지만 이 문자열을 반환하지 않는다.

Passing Arguments

리스트: 공유한 객체의 내용을 변경한다

리스트는 여러 값을 가진 변경 가능한 객체이다

numbers는 [1, 2, 3, 4, 5]라는 리스트를 가리킨다. modify2(numbers)를 호출하면 li도 같은 리스트를 참조한다.

이 예제의 리스트 +=는 원소를 추가한다

li += [100, 200]은 기존 리스트에 두 원소를 덧붙인다. 공유한 객체 자체가 변했으므로 numbers로 확인해도 늘어난 내용을 볼 수 있다.

반환값이 없어도 객체 변경은 남을 수 있다

modify2는 명시적으로 값을 반환하지 않지만 공유 리스트를 수정한다. 함수의 반환값과 함수 실행으로 발생한 객체 변경을 구분해야 한다.

호출 전후의 리스트 확인

```
def modify2(li):
    li += [100, 200]

numbers = [1, 2, 3, 4, 5]
print(numbers)
modify2(numbers)
print(numbers)
```

실행 결과

```
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 100, 200]
```

Passing Arguments

문자열과 리스트의 호출 전후 비교

In [2]:
```
msg = "Happy Birthday"
print("msg=", msg)
modify1(msg)
print("msg=", msg)
```

```
msg= Happy Birthday
msg= Happy Birthday
```

In [3]:
```
numbers = [1, 2, 3, 4, 5]
print(numbers)
modify2(numbers)
print(numbers)
```

```
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 100, 200]
```

앞서 정의한 modify1()과 modify2()를 호출한다. 문자열의 지역 이름을 다시 연결한 경우에는 바깥 값이 같고, 공유 리스트를 변경한 경우에는 바깥에서도 원소 추가가 보인다.

Passing Arguments

li += ...와 li = li + ...를 비교한다

같은 리스트 인수라도 연산이 다르면 결과가 다르다

리스트의 +=는 이 예제에서 기존 객체를 확장한다. 반면 li + [100, 200]은 새 리스트를 만들고, li = ...는 지역 이름을 그 새 객체에 연결한다.

호출한 쪽의 이름이 바뀌었는지 확인한다

두 번째 형태에서는 numbers에 새 리스트를 대입하지 않았다. 따라서 함수를 나온 뒤 numbers는 처음 리스트 [1, 2, 3]을 유지한다.

두 가지 전달 방식으로 외우지 않는다

두 함수 모두 리스트를 받았지만 바깥에서 관찰되는 결과가 다르다. 공유 객체의 변경인지 이름의 재대입인지 구분하면 차이를 설명할 수 있다.

공유 리스트를 변경하는 함수

```
def extend_items(li):
    li += [100, 200]
```

지역 이름만 다시 연결하는 함수

```
def rebind_items(li):
    li = li + [100, 200]

numbers = [1, 2, 3]
rebind_items(numbers)
print(numbers)
```

실행 결과

[1, 2, 3]

Default Parameters

인수를 생략하면 기본값을 사용한다

정의할 때 자주 사용하는 값을 정한다

rectangle_area(length=2, width=3)는 두 매개변수에 기본값을 둔다. 두 인수를 모두 생략하면 2와 3을 사용하여 넓이 6을 반환한다.

전달한 인수만 기본값을 대신한다

rectangle_area(10)은 첫 매개변수 length만 10으로 바꾼 호출이다. width는 기본값 3을 사용하므로 결과는 30이다.

기본값과 입력 위치를 함께 확인한다

rectangle_area(10, 5)는 두 값을 모두 전달하여 50을 반환한다. 기본값은 def가 실행될 때 평가되며, 이번 예제처럼 숫자 값을 두는 형태부터 사용한다.

직사각형의 넓이 함수

```
def rectangle_area(length=2, width=3):
    """직사각형의 넓이를 반환한다."""
    return length * width
```

호출마다 사용하는 값

호출	length	width	결과
()	2	3	6
(10)	10	3	30
(10, 5)	10	5	50

읽는 순서

먼저 전달한 인수와 매개변수를 연결하고, 생략한 자리에 기본값을 적용한다.

Keyword Arguments

이름을 지정하면 대응 관계가 분명해진다

위치 인수는 순서대로 연결한다

calc(10, 20, 30)은 x, y, z에 각각 10, 20, 30을 전달한다. 매개변수의 순서를 기억하고 호출해야 한다.

키워드 인수는 매개변수 이름으로 연결한다

calc(y=20, x=10, z=30)은 작성 순서가 달라도 이름이 일치하는 매개변수에 전달된다. 함수를 정의할 때 사용한 이름을 정확히 적어야 한다.

같은 자리에 두 번 전달하지 않는다

calc(10, x=20, z=30)은 x를 두 번 지정한 잘못된 호출이다. 이번 형태에서는 위치 인수를 먼저 쓰고, 이어서 키워드 인수를 사용한다.

같은 값을 전달하는 세 가지 호출

```
def calc(x, y, z):
    return x + y + z


print(calc(10, 20, 30))
print(calc(x=10, y=20, z=30))
print(calc(y=20, x=10, z=30))
```

실행 결과

```
60
60
60
```

기본값과 함께 사용

rectangle_area(width=5)는 기본 length 2를 유지하여 10을 반환한다.

Function Calls

기본값과 키워드 인수의 실행 결과

In [2]:
```
print(rectangle_area())
print(rectangle_area(10))
print(rectangle_area(10, 5))
print(rectangle_area(width=5))
```

```
6
30
50
10
```

In [3]:
```
print(calc(10, 20, 30))
print(calc(y=20, x=10, z=30))
```

```
60
60
```

앞서 정의한 rectangle_area()와 calc()를 호출한다. width만 지정하면 length는 기본값을 사용한다. 키워드 인수의 순서를 바꾸어도 연결되는 값이 같으면 결과도 같다.

Scope Rules

지역 이름과 전역 이름을 구분한다

지역 이름은 함수 호출 안에서 사용한다

매개변수와 함수 안에서 대입한 이름은 기본적으로 그 함수의 지역 이름이다. 함수 밖에서 같은 철자를 사용한다고 지역 값에 바로 접근할 수 있는 것은 아니다.

전역 이름은 이 파일이나 모듈 수준의 이름이다

함수 밖에서 정의한 r, area 등은 이 예제의 전역 이름이다. 전역이라는 말은 모든 프로그램과 모든 Notebook 커널에 자동 공유된다는 뜻이 아니다.

같은 철자도 서로 다른 이름일 수 있다

함수 안의 area와 함수 밖의 area는 별개일 수 있다. 값을 찾는 위치와 새로 대입하는 위치를 나누어 보면 예상과 다른 결과의 원인을 찾기 쉽다.

함수 안과 밖의 이름

```
def calculate_area(radius):
    result = 3.14 * radius ** 2
    return result

r = 10.0
area = calculate_area(r)
```

함수 안의 지역 이름

radius, result

함수 밖의 이름

calculate_area, r, area

Local Scope

result를 반환해도 그 지역 이름이 밖에 생기지는 않는다

함수 밖에는 반환값을 받은 이름이 있다

calculate_area(r)의 반환값을 area에 저장한다. 함수 안에서 결과를 result라는 이름으로 계산했더라도 바깥에는 그 이름이 자동 생성되지 않는다.

print(result)는 다른 이름을 찾는다

새 실행 환경에서 result를 따로 정의하지 않았다면 print(result)는 NameError를 만든다. 계산이 실패한 것이 아니라 바깥에서 잘못된 이름을 사용한 것이다.

반환값을 받은 area를 출력한다

print(area)로 바꾸면 반지름이 10.0일 때 314.0을 확인한다. 원자료와 같은 근삿값 3.14를 사용한다.

함수의 지역 이름을 밖에서 사용하는 오류

```
def calculate_area(radius):
    result = 3.14 * radius ** 2
    return result

r = float(input("Circle's radius: "))
area = calculate_area(r)
print(result)
```

10을 입력한 뒤의 오류

NameError: name 'result' is not defined

출력문 수정

print(area)

수정 후 결과

314.0

Local Scope

지역 이름 오류와 올바른 반환값 출력

In [2]:
```
def calculate_area(radius):
    result = 3.14 * radius ** 2
    return result

r = 10.0
area = calculate_area(r)
print(result)
```

```
NameError: name 'result' is not defined
```

In [3]:
```
print(area)
```

```
314.0
```

입력 10을 r = 10.0으로 재현했다. print(result)는 지역 이름을 밖에서 찾으므로 NameError가 난다. 다음 셀에서 이미 반환값을 받은 area를 출력하면 314.0이 나온다.

Global Scope

함수 안에서 전역 이름의 값을 읽기

함수 밖의 값을 읽는 형태도 가능하다

이 함수는 매개변수가 없지만 계산식에서 r을 사용한다. 함수가 실행되는 시점에 전역 이름 r이 있어야 해당 값을 찾아 넓이를 계산한다.

정의 시점과 호출 시점을 나누어 본다

def를 실행할 때 본문의 계산을 하지 않으므로 r은 정의 뒤에 대입해도 된다. 단, calculate_area()를 호출하기 전에는 r에 값이 연결되어야 한다.

함수의 입력이 코드 밖에 숨어 있을 수 있다

calculate_area()만 보면 어떤 반지름을 사용하는지 드러나지 않는다. calculate_area(r)처럼 매개변수를 사용하는 형태와 비교하면 의존하는 값이 더 명확해진다.

전역 r을 읽는 예제

```
def calculate_area():
    result = 3.14 * r ** 2
    return result

r = 10.0
area = calculate_area()
print(area)
```

실행 결과

```
314.0
```

예제의 입력

화면의 r = 10.0은 사용자가 반지름 10을 입력한 경우에 해당한다. 다른 r을 대입하고 다시 호출하면 결과도 달라진다.

Global Scope

함수 안의 area에 대입했는데 왜 바깥은 0일까?

전역 area를 바꾸려는 의도와 실제 동작은 다르다

함수 안의 area = ...는 별도의 지역 이름 area를 만든다. 같은 철자를 사용했다는 이유만으로 전역 이름에 대입되는 것은 아니다.

지역 계산 결과를 반환하지 않고 끝낸다

현재 return에는 값이 없다. 따라서 호출 결과는 None이며, 바깥 코드에서도 계산 결과를 다른 이름에 대입하지 않는다.

전역 area는 처음 값 0을 유지한다

함수 밖의 area = 0 이후에 전역 이름을 바꾸는 명령이 없다. 내부에서 314.0을 계산해도 마지막 print(area)는 0을 출력한다.

지역 area와 전역 area가 분리된 예

```
def calculate_area(radius):
    area = 3.14 * radius ** 2
    return

area = 0
r = 10.0
calculate_area(r)
print(area)
```

실행 결과

```
0
```

구분할 두 값

```
함수 안의 지역 area: 314.0
함수 밖의 전역 area: 0
```

Global Scope

global을 쓰는 방식과 반환값을 받는 방식

global은 대입할 이름의 범위를 지정한다

함수 안에서 global area를 선언하면 이어지는 area 대입은 해당 모듈의 전역 이름을 대상으로 한다. 함수가 바깥의 상태를 직접 바꾸는 방식이다.

전역 이름에 직접 대입

```
def calculate_area(radius):
    global area
    area = 3.14 * radius ** 2

area = 0
calculate_area(10.0)
print(area)  # 314.0
```

return으로 결과를 받아도 목적을 달성할 수 있다

함수는 넓이를 반환하고 호출한 코드가 area에 대입하도록 만들 수 있다. 입력과 반환값이 드러나므로 계산 결과의 이동을 추적하기 쉽다.

반환값을 받아 대입하는 다른 형태

```
def calculate_area(radius):
    return 3.14 * radius ** 2

area = calculate_area(10.0)
print(area)  # 314.0
```

Global Scope

같은 계산에서도 바깥 이름을 바꾸는 경로는 다르다

In [2]:
```
area = 0
local_version(10.0)
print(area)
```

```
0
```

In [3]:
```
global_version(10.0)
print(area)
```

```
314.0
```

In [4]:
```
area = return_version(10.0)
print(area)
```

```
314.0
```

local_version, global_version, return_version은 앞서 비교한 세 함수 형태이다. 지역 대입 뒤에는 0, 전역 대입과 반환값을 받은 대입 뒤에는 각각 314.0을 확인한다.

Returning Multiple Values

여러 값을 반환할 때는 묶음으로 돌려준다

쉼표로 구분한 값은 튜플로 묶인다

return 1, 2, 3은 세 개의 값을 담은 튜플 하나를 반환한다. 여러 결과를 한 번에 전달할 수 있지만 반환 객체가 서로 독립적으로 세 번 전달되는 것은 아니다.

호출한 쪽에서 순서대로 풀어 받을 수 있다

a, b, c = sub()는 반환된 튜플의 값을 순서대로 각 이름에 대입한다. 이 형태에서는 받을 이름의 개수와 값의 개수가 맞아야 한다.

하나의 이름으로 전체를 받을 수도 있다

values = sub()처럼 쓰면 튜플 전체가 values에 연결된다. 하나로 받을지 여러 이름으로 나누어 받을지는 호출한 코드에서 정한다.

세 값을 묶어 반환하기

```
def sub():
    return 1, 2, 3

a, b, c = sub()
print(a, b, c)

values = sub()
print(values)
print(type(values))
```

실행 결과

```
1 2 3
(1, 2, 3)
<class 'tuple'>
```

Returning Multiple Values

튜플 전체와 풀어 받은 값을 비교하기

In [1]:
```
def sub():
    return 1, 2, 3
```

In [2]:
```
values = sub()
print(values)
print(type(values))
```

```
(1, 2, 3)
<class 'tuple'>
```

In [3]:
```
a, b, c = sub()
print(a, b, c)
```

```
1 2 3
```

sub()는 튜플 (1, 2, 3)을 반환한다. values로 전체를 받거나 a, b, c에 순서대로 풀어 받을 수 있다. 두 경우 모두 같은 반환값을 사용한다.

String Formatting

str.format()으로 문자열에 값을 배치한다

중괄호는 값을 넣을 위치이다

"{0} {1}".format(company_name, price)에서 0과 1은 전달된 값의 순서를 뜻한다. 두 값 사이의 공백은 형식 문자열에 직접 적은 문자이다.

위치 번호 대신 이름을 사용할 수도 있다

"{0} {price}".format(company_name, price=50000)은 첫 위치 인수와 이름을 지정한 값을 함께 배치한다. 기존 변수 price의 내용과 호출의 키워드 값을 구분한다.

형식화는 새 문자열을 만든다

결과를 stock_price에 저장해 사용할 수 있다. 여기의 회사명과 가격은 문자열 포맷팅을 위한 원자료의 예시이며 실제 현재 시세를 나타내지 않는다.

위치 번호로 값 배치

```
company_name = "Samsung"
price = "85,000"
stock_price = "{0} {1}".format(
    company_name, price)
print(stock_price)
```

실행 결과

```
Samsung 85,000
```

이름으로 값 지정

```
print("{0} {price}".format(
    company_name, price=50000))
```

실행 결과

```
Samsung 50000
```

String Formatting

f-string은 중괄호 안의 표현식을 계산한다

문자열 앞에 f를 붙인다

f"{company_name} {price}"는 현재 이름의 값을 문자열 안에 배치한다. 일반 문자열 "{price}"와 달리 중괄호 부분을 실제 값으로 채운다.

문자열을 만드는 시점의 값을 사용한다

price가 나중에 바뀌어도 이미 만든 stock_price는 자동 갱신되지 않는다. 새 값을 반영하려면 f-string을 만드는 문장을 다시 실행한다.

여러 줄의 문자열도 만들 수 있다

세 개의 따옴표나 \n으로 여러 줄 문자열을 만든다. print()는 실제 줄바꿈을 보여 준다. 문자열의 값과 표시 방식을 구분한다.

변수 값을 포함하는 문자열

```
company_name = "Samsung"
price = "85,000"
stock_price = f"{company_name} {price}"
print(stock_price)
```

실행 결과

```
Samsung 85,000
```

줄바꿈을 포함한 다른 예

```
new_price = "100,000"
text = f"{company_name}\n{price}\n{new_price}"
print(text)
```

실행 결과

Samsung, 85,000, 100,000이 각각 다른 줄에 출력된다.

String Formatting

format()과 f-string의 실제 출력 비교

In [1]:
```
company_name = "Samsung"
price = "85,000"
print("{0} {1}".format(company_name, price))
print(f"{company_name} {price}")
```

```
Samsung 85,000
Samsung 85,000
```

In [2]:
```
new_price = "100,000"
text = f"{company_name}\n{price}\n{new_price}"
print(text)
```

```
Samsung
85,000
100,000
```

format()과 f-string으로 같은 문자열을 만들 수 있다. 마지막 셀에서는 줄바꿈을 포함한 문자열을 출력한다. 회사명과 가격은 원자료의 문자열 배치 연습용 예시이다.

Number Formatting

소수점 표시와 원래 값을 구분한다

콜론 뒤에 표시 형식을 지정한다

:d는 정수를 십진수로 표시하고 :.2f는 소수점 아래 두 자리로 표시한다. 형식 문자열은 값의 순서뿐 아니라 결과의 표현 방식도 지정할 수 있다.

표시 자릿수는 값 자체를 바꾸지 않는다

2 ** 0.5를 :.2f로 표시하면 1.41이다. 하지만 원래 계산한 값이 1.41로 대입되거나 저장되는 것은 아니므로 뒤의 계산은 원래 값을 사용한다.

너비와 정렬도 선택할 수 있다

{name:>10}은 최소 너비 10에서 오른쪽 정렬한다. ^는 가운데 정렬이다. 값이 너비보다 길다고 자동으로 잘리지는 않는다.

정수와 실수의 형식 지정

```
c, d = 3, 4
e = c + d
print("e = {0:d}".format(e))
print("{0:d}, {1:.2f}".format(
    c ** 5, 2 ** 0.5))
```

실행 결과

```
e = 7
243, 1.41
```

같은 자릿수를 f-string으로 지정

```
value = 2 ** 0.5
print(f"{value:.2f}")
print(value)
```

Number Formatting

표시를 바꿔도 계산 값은 그대로 남는다

In [1]:
```
c, d = 3, 4
e = c + d
print("e = {0:d}".format(e))
print("{0:d}, {1:.2f}".format(c ** 5, 2 ** 0.5))
```

```
e = 7
243, 1.41
```

In [2]:
```
value = 2 ** 0.5
print(f"{value:.2f}")
print(value)
```

```
1.41
1.4142135623730951
```

정수와 실수의 표시 형식을 지정한 출력이다. 마지막의 value는 같은 값인데도 :.2f로 표시하면 1.41, 원래 값을 출력하면 더 많은 소수 자릿수가 보인다.

Practice

종합 실습: 넓이를 반환하고 보기 좋게 출력하기

입력, 계산 함수, 출력을 나누어 작성한다

calculate_area(radius)는 반지름을 받아 원의 넓이를 반환한다. 입력과 출력은 함수 밖에 두어 함수가 계산 결과만 돌려주도록 작성한다.

반환값을 받은 이름을 이용한다

함수 안의 result를 밖에서 출력하지 않는다. 반환값을 area에 저장하고 f-string에서 소수점 아래 두 자리로 표시한다.

정답을 실행하기 전에 예상한다

반지름 10의 계산 값은 314.0이고 표시 결과는 314.00이다. global을 사용하지 않고도 바깥에서 결과를 사용할 수 있는 이유를 설명한다.

완성할 코드

```
def calculate_area(radius):
    result = 3.14 * radius ** 2
    _____ result

r = float(input("반지름: "))
area = ____________________
print(f"넓이: {area:____}")
```

검증할 입력과 출력

입력 10 → 넓이: 314.00
입력 2 → 넓이: 12.56

확인 질문

인수와 매개변수는 무엇인가?
왜 result 대신 area를 출력하는가?

Chapter Overview

함수의 입력, 반환, 범위를 연결해서 설명한다

정의와 호출의 차이

def는 재사용할 함수를 준비하고, 호출은 본문을 실행한다. 호출할 때 전달한 인수는 매개변수에 연결되며, return이 돌려준 값으로 바깥 대입이 완성된다.

이름과 객체의 변화

지역 이름을 재대입하는 것과 공유 객체를 변경하는 것은 다르다. 전역 이름을 직접 바꾸는 global과 계산 결과를 return하는 방법도 구분한다.

형식을 바꾸는 단계

기본값과 키워드 인수로 호출을 명확하게 작성한다. 결과는 튜플로 묶어 전달하거나 format()과 f-string으로 표현할 수 있다. 표시 방식과 원래 값을 구분한다.

종합 실습의 완성 코드

```
def calculate_area(radius):
    result = 3.14 * radius ** 2
    return result

r = float(input("반지름: "))
area = calculate_area(r)
print(f"넓이: {area:.2f}")
```

10을 입력한 결과

넓이: 314.00

설명할 연결 관계

인수 r → 매개변수 radius
지역 result의 값 → 바깥 area
area → 형식화한 출력 문자열